

Chapitre 5 : Perceptron Multicouche (MLP)

Apprentissage Supervisé

Dr Seydou Nourou SYLLA

Université Alioune DIOP de Bambey
Master SI/SR

2 mars 2026

Plan du chapitre

- 1 Du perceptron au MLP
- 2 Forward propagation
- 3 Backpropagation
- 4 Implémentation from scratch
- 5 Hyperparamètres et bonnes pratiques

Le neurone artificiel (perceptron)

Inspiration biologique : mimique un neurone.

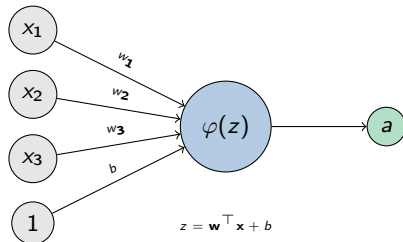
Calcul d'un neurone :

❶ **Combinaison linéaire** des entrées :

$$z = \sum_{j=1}^p w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$$

❷ **Fonction d'activation** :

$$a = \varphi(z)$$



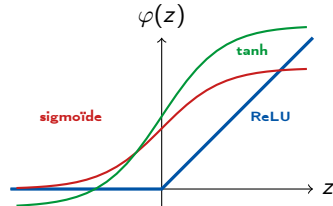
Cas particulier : Si $\varphi = \sigma$ (sigmoïde) $\Rightarrow$ régression logistique !

Fonctions d'activation

Nom	Formule	Usage
Sigmoïde	$\frac{1}{1+e^{-z}}$	Sortie binaire
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	Cachées (historique)
ReLU	$\max(0, z)$	Cachées (standard)
Leaky ReLU	$\max(0.01z, z)$	ReLU amélioré
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	Sortie multi-classes

Pourquoi ReLU est populaire :

- Gradient constant = 1 (pas de saturation)
- Calcul simple : $\max(0, z)$
- Accélère la convergence



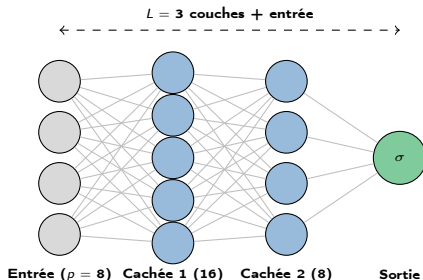
Architecture d'un MLP

MLP = empilement de couches :

- 1 **Couche d'entrée** : p neurones (features)
- 2 **Couches cachées** : transformations non-linéaires
- 3 **Couche de sortie** : prédiction finale

Notation :

- L : nombre de couches
- $n^{[l]}$: nb de neurones à la couche l
- $W^{[l]}$: matrice de poids de la couche l
- $b^{[l]}$: biais de la couche l



Forward propagation – Calcul couche par couche

Pour chaque couche $l = 1, \dots, L$:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \quad (\text{combinaison linéaire})$$

$$a^{[l]} = \varphi^{[l]}(z^{[l]}) \quad (\text{activation})$$

avec $a^{[0]} = x$ (entrée).

Pour notre IDS (architecture $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$) :

$$z^{[1]} = W^{[1]}x + b^{[1]} \in \mathbb{R}^{16}$$

$$a^{[1]} = \text{ReLU}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \in \mathbb{R}^8$$

$$a^{[2]} = \text{ReLU}(z^{[2]})$$

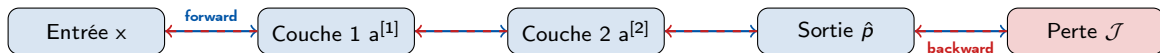
$$z^{[3]} = W^{[3]}a^{[2]} + b^{[3]} \in \mathbb{R}$$

$$\hat{p} = \sigma(z^{[3]}) \in (0, 1)$$

Rétropropagation – Intuition

Problème : Comment calculer $\frac{\partial \mathcal{J}}{\partial W^{[l]}}$ pour chaque couche l ?

Réponse : La **règle de la chaîne** (chain rule) appliquée de la couche de sortie vers l'entrée.



Erreur de la couche de sortie (log-loss binaire) :

$$\delta^{[L]} = \hat{p} - y$$

Rétropropagation vers la couche l :

$$\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \odot \varphi'^{[l]}(z^{[l]})$$

Gradients et mise à jour des poids

Gradients pour la couche l :

$$\frac{\partial \mathcal{J}}{\partial \mathbf{W}^{[l]}} = \frac{1}{n} \delta^{[l]} (\mathbf{a}^{[l-1]})^\top \quad \frac{\partial \mathcal{J}}{\partial \mathbf{b}^{[l]}} = \frac{1}{n} \sum_i \delta_i^{[l]}$$

Mise à jour :

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \eta \cdot \frac{\partial \mathcal{J}}{\partial \mathbf{W}^{[l]}} \quad \mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \eta \cdot \frac{\partial \mathcal{J}}{\partial \mathbf{b}^{[l]}}$$

Dérivées des fonctions d'activation

ReLU :

Tanh :

Sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

Hyperparamètres critiques du MLP

Hyperparamètre	Impact	Valeur IDS
Architecture ($n^{[1]}$, $n^{[2]}$, ...)	Capacité du modèle	(32, 16)
Activation cachées	Non-linéarité	ReLU
Learning rate η	Vitesse convergence	0.001–0.01
Batch size	Bruit gradient / vitesse	64–256
Nb d'époques	Convergence	50–200
Dropout (p_d)	Régularisation	0.2–0.5
Initialisation	Symétrie brisée	He (ReLU)

Problème de la symétrie

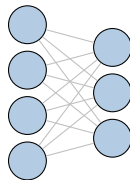
Si tous les poids sont initialisés à 0, tous les neurones calculent la même chose $\Rightarrow$ réseau équivalent à 1 neurone. L'initialisation aléatoire **brise la symétrie**.

Dropout (Srivastava et al., 2014) :

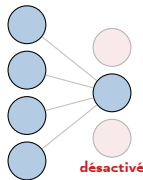
- Pendant l'entraînement, désactiver aléatoirement chaque neurone avec probabilité p_d
- Force le réseau à apprendre des représentations **redondantes**
- À l'inférence : utiliser tous les neurones, multiplier par $(1 - p_d)$

Implémentation :

Sans Dropout



Avec Dropout



Forces et faiblesses du MLP

Forces :

- + Approximateur universel (th. de Cybenko)
- + Capture les non-linéarités complexes
- + Prédiction probabilistes
- + Extensible : CNN, RNN, Transformer
- + Excellentes performances sur grands datasets

Faiblesses :

- Boîte noire (peu interprétable)
- Nombreux hyperparamètres à régler
- Nécessite beaucoup de données
- Normalisation obligatoire
- Entraînement coûteux (GPU souvent nécessaire)

Modèle	Acc. test	Interprétabilité	Données nécessaires
LR	97.1%	Élevée	Peu
RF	99.8%	Moyenne	Moyen
MLP	99.7%	Faible	Beaucoup

À retenir

- MLP = empilement de transformations linéaires + activations non-linéaires
- L'entraînement se fait par **forward pass** (calcul) + **backward pass** (gradients)
- ReLU est l'activation standard pour les couches cachées
- Dropout est la technique de régularisation principale
- Le MLP est la brique de base des réseaux de neurones profonds (Deep Learning)

Hyperparamètre	Recommandé IDS
Architecture	$[p, 32, 16, 1]$ ou $[p, 64, 32, 1]$
Activation cachées	ReLU
Initialisation	He
Learning rate	0.001 (Adam)
Batch size	128–256
Dropout	0.2 (si overfitting)